AMERICAN INTERNATIONAL UNIVERSITY BANGLADESH (AIUB)

FACULTY OF SCIENCE & TECHNOLOGY

Course Title

INTRODUCTION TO DATABASE (CSC2108)

Semester:

Section: [M]

TITLE

Online food delivery management system

Supervised By

JUBAYER AHAMED

Submitted By: Group no: 09

Name	ID
Fahim Ahmed	23-55423-3
Sajal Kumar Ghosh	23-55419-3
Anika Tabassum	23-55070-3
Md. Towhidul Islam	23-55036-3

TABLE OF CONTENTS

Introduction

The “Online Food Delivery Management System” is an all-inclusive platform aimed at optimizing the food ordering and delivery experience. This system links customers, restaurants, and delivery personnel, ensuring an effective and user-friendly interface for everyone involved. Designed with a strong database structure, it facilitates smooth interactions among the key entities: customers, orders, delivery personnel, restaurants, and menus.

The primary goal of this system is to enable customers to browse menus, place orders, and have food delivered to their doorstep efficiently. Customers can register their details, including their name, Phone number, and address (City, Road, and house number), and explore a wide range of food options available on the menu. Each menu item is categorized with attributes such as food name, Category, Price, and a unique food ID to ensure easy browsing.

Once an order is placed, it is linked to the customer and processed by the associated restaurant. Restaurants in the system manage their operations by maintaining essential details such as their name, location (City, Road, and ZIP), helpline number, and a unique restaurant ID. Restaurants handle orders and prepare food for delivery, ensuring a high level of service quality.

The delivery partner plays a vital role in bridging the gap between restaurants and customers. Delivery personnel details, including rider name, Phone number, Rider Vehicle information, and company name, are maintained in the system. Each order is assigned to a delivery partner, who ensures Order Timely and accurate delivery to the customer.

By including essential functions like order monitoring, real-Order Time Updates, and safe data processing, this project highlights the significance of managing databases for modern food services. The Online Food Delivery Management System provides a dependable and effective solution for food delivery operations by promoting efficient communication among the organizations, meeting the increasing need for digital comfort and accessibility.

Case Study / Scenario

An online food delivery management system makes it easy for customers to order food from their favorite restaurants and get it delivered to their homes. The customers Has a name, address, and Phone number. The address has three parts: City, Road, house number. Also, Customers register with unique ID. They can browse the menu offered by various restaurants, view details such as food names, categories, unique FoodID and Prices, and place multiple orders. Order has OrderID, Bill, OrderDate and OrderTime. The delivery partner is responsible for ensuring the order reaches the customers. The delivery partner has a unique ID, company name, rider name, rider contact number, and Rider Vehicle. When a customer places an order at a restaurant, the restaurant prepares the orders, and the delivery partner ensures it reaches the customer. Restaurants, each identified by a unique ID, restaurant name, helpline and location. The restaurant location is composed of City, zip code, Road.

ER Diagram

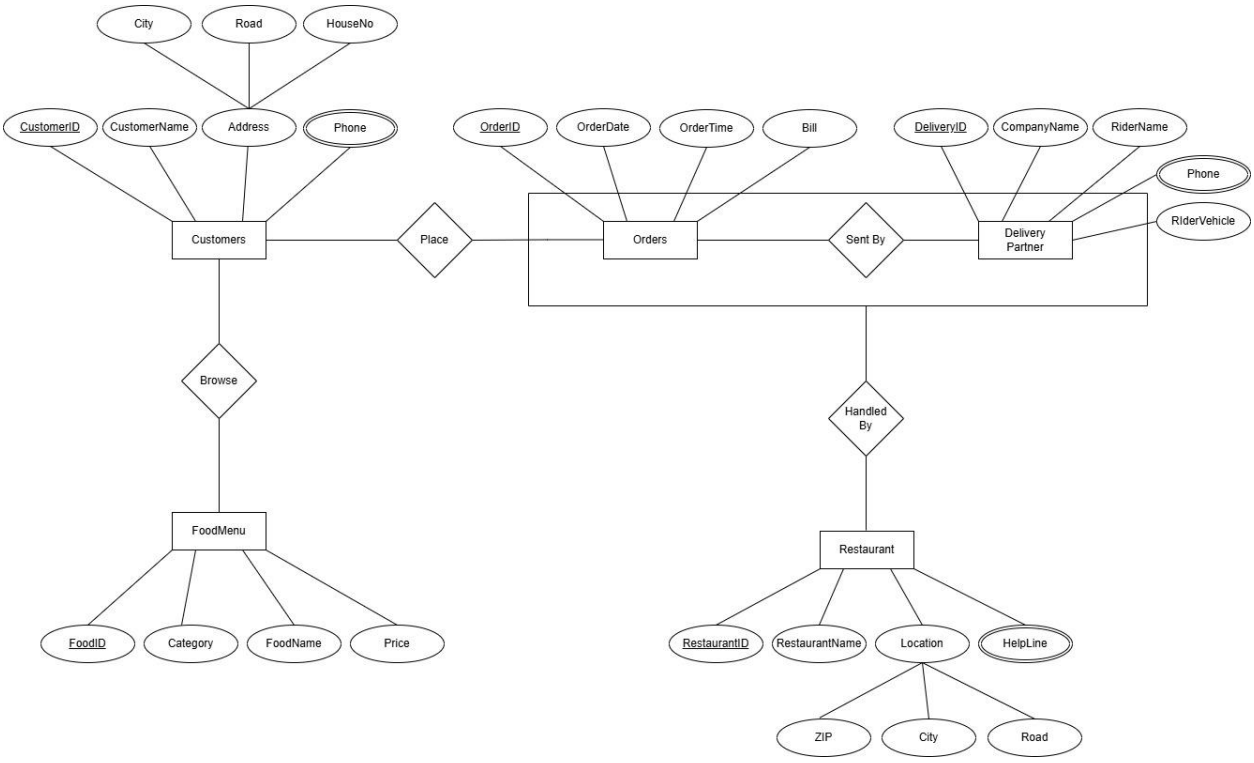


Fig: Online Food Delivery Management System

Normalization

Browse Relation

UNF:

CustomerID, CustomeName, Phone, Address, City, Road, HouseNo, FoodID, FoodName, Category, Price

1NF:

CustomerID, CustomeName, Phone, City, Road, HouseNo, FoodID, FoodName, Category, Price

2NF:

(1) CustomerID (PK), FoodID (FK), CustomeName, Phone, City, Road, HouseNo

(2) FoodID (PK), FoodName, Category, Price

3NF:

(1) FoodID (PK), FoodName, Category, Price

(2) CustomerID (PK), FoodID (FK) , CustomeName, Phone, City

(3) City, Road, HouseNo

Place Relation

UNF:

CustomerID, CustomeName, Phone, Address, City, Road, HouseNo, OrderID, OrderTime, OrderDate, Bill

1NF:

CustomerID, CustomeName, Phone, City, Road, HouseNo, OrderID, OrderTime, OrderDate, Bill

2NF:

(1) CustomerID(PK), CustomeName, Phone, City, Road, HouseNo

(2) OrderID(PK), CustomerID(FK) , OrderTime, OrderDate, Bill

3NF:

(1) OrderID(PK), CustomerID(FK) , OrderTime, OrderDate, Bill

(2) CustomerID(PK), CustomeName, Phone, City

(3) City, Road, HouseNo

Sent By Relation

UNF:

OrderID, OrderTime, Bill, OrderDate, DeliveryID, RiderName, CompanyName, Phone, RiderVehicle

1NF:

OrderID, OrderTime, Bill, OrderDate, DeliveryID, RiderName, CompanyName, Phone, RiderVehicle

2NF:

(1) OrderID(PK), DeliveryID(FK), OrderTime, Bill, OrderDate

(2) DeliveryID(PK), CompanyName, RiderName, Phone, RiderVehicle

3NF:

(1) OrderID(PK), DeliveryID(FK), OrderTime, Bill, OrderDate

(2) DeliveryID(PK), CompanyName, RiderName, Phone, RiderVehicle

Handled BY Relation

UNF:

OrderID, OrderTime, Bill, OrderDate, DeliveryID, CompanyName, RiderName, Phone, RiderVehicle, RestaurantID, RestaurantName , Helpline , Location, City, Zip, Road

1NF:

OrderID, OrderTime, Bill, OrderDate, DeliveryID, CompanyName, RiderName, Phone, RiderVehicle, RestaurantID, RestaurantName , Helpline , City, Zip, Road

2NF:

(1) OrderID(PK), OrderTime, Bill, OrderDate

(2) DeliveryID(PK), CompanyName, RiderName, Phone, RiderVehicle,

(3) RestaurantID(PK), OrderID(FK) , RestaurantName , Helpline , City, Zip, Road

3NF:

(1) OrderID(PK), OrderTime, Bill, OrderDate

(2) DeliveryID(PK), CompanyName, RiderName, Phone, RiderVehicle,

(3) RestaurantID(PK), OrderID(FK) ,RestaurantName , Helpline , City

(4) City, Zip, Road

Finalization

(1) FoodID (PK), FoodName, Category, Price

(2) CustomerID (PK), FoodID (FK) , CustomeName, Phone, City

(3) City, Road, HouseNo

(4) OrderID(PK), CustomerID(FK) , OrderTime, OrderDate, Bill

(5) CustomerID(PK), CustomeName, Phone, City

~~(6) City, Road, HouseNo~~

(7) OrderID(PK), DeliveryID(FK), OrderTime, Bill, OrderDate

(8) DeliveryID(PK), CompanyName, RiderName, Phone, RiderVehicle

(9) OrderID(PK), OrderTime, Bill, OrderDate

(10) ~~DeliveryID(PK), CompanyName, RiderName, Phone, RiderVehicle~~

(11) RestaurantID(PK), OrderID(FK) ,RestaurantName , Helpline , City

(12) City, Zip, Road

Table Creation (DDL Operations)

StudentID1: 23-55423-3 Name: Fahim Ahmed	StudentID3: 23-55070-3 Name: Anika Tabassum
StudentID2: 23-55419-3 Name: Sajal Kumar Ghosh	StudentID4: 23-55036-3 Name: Md. Towhidul Islam
CO4: Creating DML, DDL using Oracle and connection with ODBC/JDBC for existing JAVA application	
PO-e-2: Use modern engineering and IT tools for prediction and modeling of complex computer science and engineering problem	Marks

1. FoodMenu

```
CREATE TABLE FoodMenu (FoodID NUMBER PRIMARY KEY, FoodName
VARCHAR2(20) NOT NULL, Category VARCHAR2(20), Price NUMBER(10) NOT NULL);

DESC FoodMenu;
```

Results Explain Describe Saved SQL History

Object Type **TABLE** Object **FOODMENU**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
FOODMENU	FOODID	Number	-	-	-	1	-	-	-
	FOODNAME	Varchar2	20	-	-	-	-	-	-
	CATEGORY	Varchar2	20	-	-	-	✓	-	-
	PRICE	Number	-	10	0	-	-	-	-

1 - 4

Fig: FoodMenu Table Creation

2. Customers

Autocommit Display 10

```
CREATE TABLE Customers (CustomerID NUMBER PRIMARY KEY, CustomerName VARCHAR2(20) NOT NULL, Phone
NUMBER(15) UNIQUE NOT NULL, City VARCHAR2(20));
DESC Customers;
```

Results Explain Describe Saved SQL History

Object Type **TABLE** Object **CUSTOMERS**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
CUSTOMERS	CUSTOMERID	Number	-	-	-	1	-	-	-
	CUSTOMERNAME	Varchar2	20	-	-	-	-	-	-
	PHONE	Number	-	15	0	-	-	-	-
	CITY	Varchar2	20	-	-	-	✓	-	-

1 - 4

Fig: Customers Table Creation

3.Orders

Autocommit Display 10 Save

```
CREATE TABLE Orders (OrderID NUMBER PRIMARY KEY ,OrderTime VARCHAR2(20) NOT NULL,OrderDate VARCHAR2(20) NOT NULL,Bill NUMBER(15) NOT NULL);

DESC Orders;
```

Results Explain Describe Saved SQL History

Object Type **TABLE** Object **ORDERS**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
ORDERS	ORDERID	Number	-	-	-	1	-	-	-
	ORDERTIME	Varchar2	20	-	-	-	-	-	-
	ORDERDATE	Varchar2	20	-	-	-	-	-	-
	BILL	Number	-	15	0	-	-	-	-

1 - 4

Fig: Orders Table Creation

4.Choice

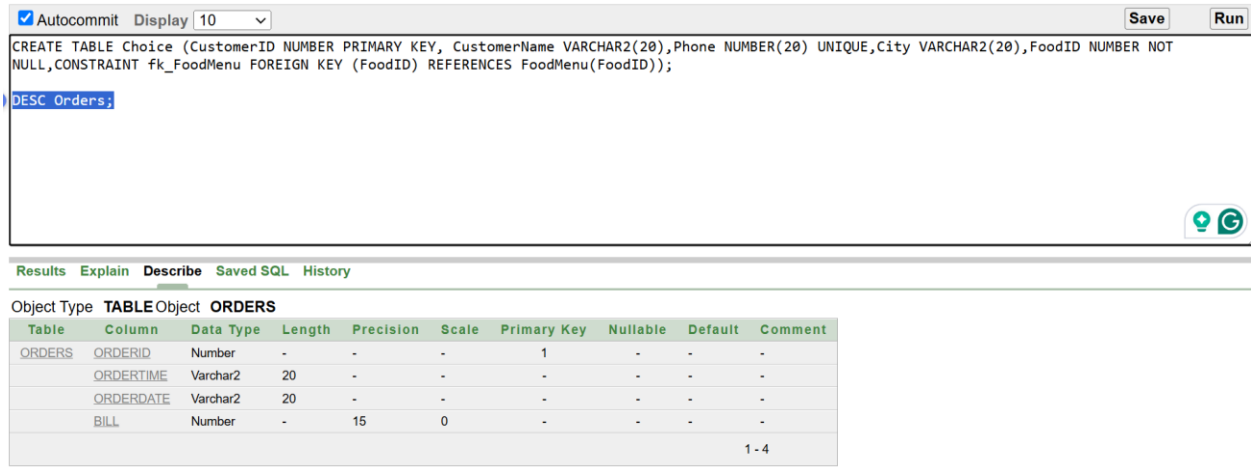


Fig: Choice Table Creation

5. Address

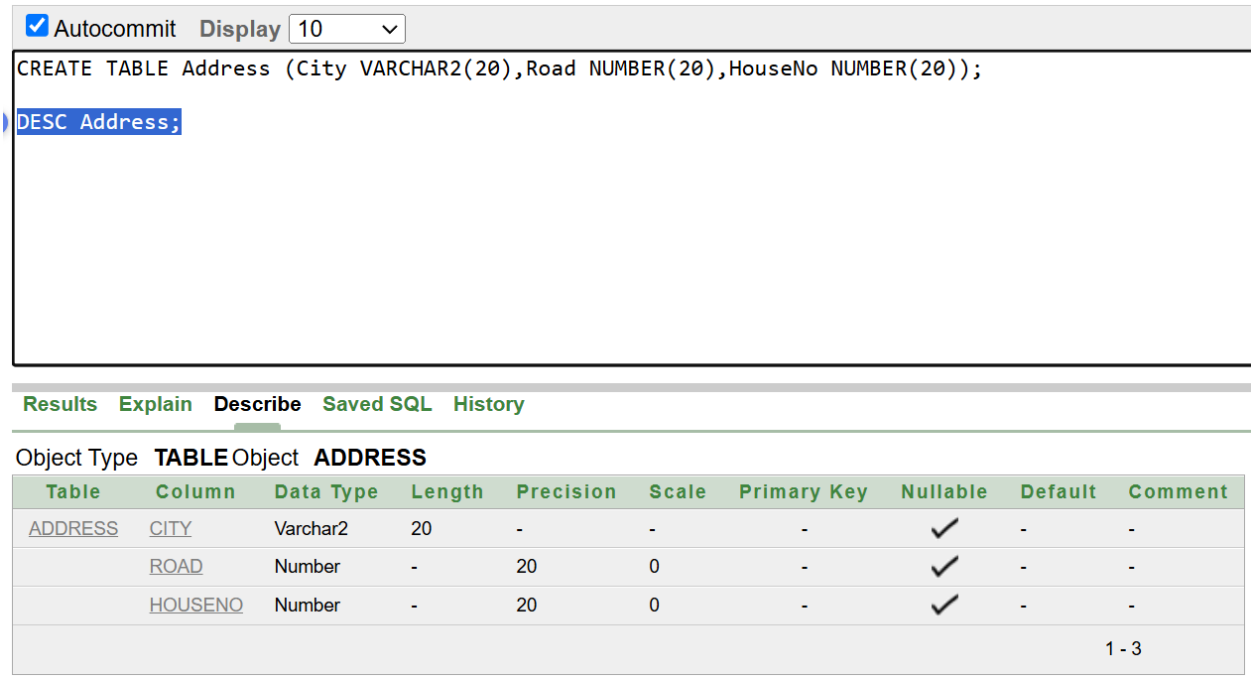


Fig: Address Table Creation

6.DeliveryPartner

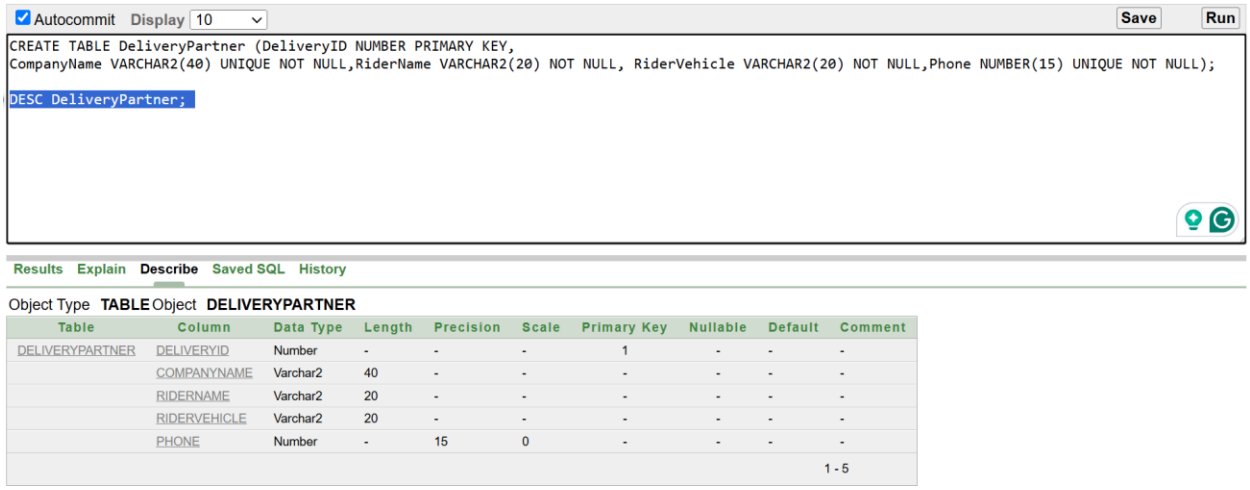


Fig: DeliveryPartner Table Creation

7.Location

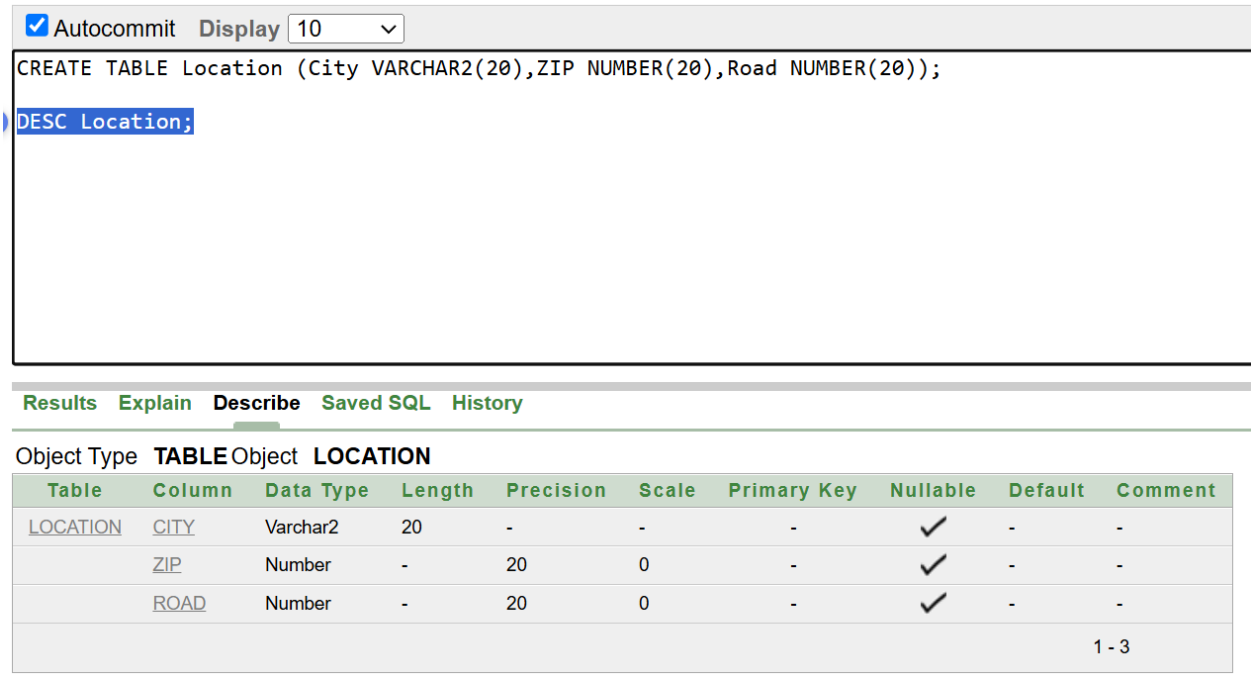


Fig: Location Table Creation

8.OrdersPending

Autocommit Display 10 Save Run

```
CREATE TABLE OrdersPending (OrderID NUMBER PRIMARY KEY,CustomerID NUMBER NOT NULL,OrderTime VARCHAR2(20) NOT NULL,OrderDate VARCHAR2(20) NOT
NULL,Bill NUMBER(15) NOT NULL,CONSTRAINT fk_Customer FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID));

DESC OrdersPending;
```

Results Explain Describe Saved SQL History

Object Type **TABLE** Object **ORDERSPENDING**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
ORDERSPENDING	ORDERID	Number	-	-	-	1	-	-	-
	CUSTOMERID	Number	-	-	-	-	-	-	-
	ORDERTIME	Varchar2	20	-	-	-	-	-	-
	ORDERDATE	Varchar2	20	-	-	-	-	-	-
	BILL	Number	-	15	0	-	-	-	-

1 - 5

Fig: OrdersPending Table Creation

9. OrderProcess

Autocommit Display 10 Save Run

```
CREATE TABLE OrderProcess (OrderID NUMBER PRIMARY KEY,DeliveryID NUMBER NOT NULL,OrderTime VARCHAR2(20) NOT NULL,OrderDate VARCHAR2(20) NOT
NULL,Bill NUMBER(15) NOT NULL,CONSTRAINT fk_DeliveryPartner FOREIGN KEY (DeliveryID) REFERENCES DeliveryPartner(DeliveryID));

desc OrderProcess;
```

Results Explain Describe Saved SQL History

Object Type **TABLE** Object **ORDERPROCESS**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
ORDERPROCESS	ORDERID	Number	-	-	-	1	-	-	-
	DELIVERYID	Number	-	-	-	-	-	-	-
	ORDERTIME	Varchar2	20	-	-	-	-	-	-
	ORDERDATE	Varchar2	20	-	-	-	-	-	-
	BILL	Number	-	15	0	-	-	-	-

1 - 5

Fig: OrderProcess Table Creation

10. Restaurant

Autocommit Display 10 Save Run

```
CREATE TABLE Restaurant (RestaurantID NUMBER PRIMARY KEY,RestaurantName VARCHAR2(30) NOT NULL,HelpLine NUMBER(15) UNIQUE NOT NULL,City
VARCHAR2(10),OrderID NUMBER NOT NULL,CONSTRAINT fk_Orders FOREIGN KEY (OrderID) REFERENCES Orders(OrderID));

DESC Restaurant;
```

Results Explain Describe Saved SQL History

Object Type **TABLE** Object **RESTAURANT**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
RESTAURANT	RESTAURANTID	Number	-	-	-	1	-	-	-
	RESTAURANTNAME	Varchar2	30	-	-	-	-	-	-
	HELPLINE	Number	-	15	0	-	-	-	-
	CITY	Varchar2	10	-	-	-	✓	-	-
	ORDERID	Number	-	-	-	-	-	-	-

1 - 5

Fig: Restaurant Table Creation

Inserted Values in the tables

1. FoodMenu

```
INSERT INTO FoodMenu VALUES (1, 'Burger', 'Fast Food', 300);
INSERT INTO FoodMenu VALUES (2, 'Pasta', 'Casual Dining', 250);
INSERT INTO FoodMenu VALUES (3, 'Steak', 'Fine Dining', 2000);
INSERT INTO FoodMenu VALUES (4, 'Sushi', 'Ethnic Cuisine', 600);
INSERT INTO FoodMenu VALUES (5, 'Vegan Bowl', 'Specialty Cuisine', 199);

SELECT * FROM FoodMenu;
```

Results | Explain | Describe | Saved SQL | History

FOODID	FOODNAME	CATEGORY	PRICE
1	Burger	Fast Food	300
2	Pasta	Casual Dining	250
3	Steak	Fine Dining	2000
4	Sushi	Ethnic Cuisine	600
5	Vegan Bowl	Specialty Cuisine	199

5 rows returned in 0.01 seconds CSV Export

Fig: Data Insertion In FoodMenu Table

2. Customers

```
INSERT INTO Customers VALUES (101, 'Fahim', 01987654321, 'Dhaka');
INSERT INTO Customers VALUES (102, 'Anika', 01987654322, 'Dhaka');
INSERT INTO Customers VALUES (103, 'Sajal', 01987654323, 'Dhaka');
INSERT INTO Customers VALUES (104, 'Towhid', 01987654324, 'Dhaka');
INSERT INTO Customers VALUES (105, 'Anchol', 01987654325, 'Dhaka');

SELECT * FROM Customers;
```

Results | Explain | Describe | Saved SQL | History

CUSTOMERID	CUSTOMERNAME	PHONE	CITY
101	Fahim	1987654321	Dhaka
102	Anika	1987654322	Dhaka
103	Sajal	1987654323	Dhaka
104	Towhid	1987654324	Dhaka
105	Anchol	1987654325	Dhaka

Fig: Data Insertion In Customers Table

3. Orders

Autocommit Display 10

```
INSERT INTO Orders VALUES (10001, '9 P.M.','01-JAN-2025',250);
INSERT INTO Orders VALUES (10002, '10 P.M.','23-JAN-2025',2000);
INSERT INTO Orders VALUES (10003, '8.30 P.M.','19-JAN-2025',199);
INSERT INTO Orders VALUES (10004, '7 P.M.','01-FEB-2025',600);
INSERT INTO Orders VALUES (10005, '2 P.M.','31-JAN-2025',300);

SELECT * FROM Orders;
```

Results Explain Describe Saved SQL History

ORDERID	ORDERTIME	ORDERDATE	BILL
10001	9 P.M.	01-JAN-2025	250
10002	10 P.M.	23-JAN-2025	2000
10003	8.30 P.M.	19-JAN-2025	199
10004	7 P.M.	01-FEB-2025	600
10005	2 P.M.	31-JAN-2025	300

Fig: Data Insertion In Orders Table

4.Choice

Autocommit Display 10

```
INSERT INTO Choice VALUES (101, 'Fahim', 01987654321, 'Dhaka',2);
INSERT INTO Choice VALUES (102, 'Anika', 01987654322, 'Dhaka',3);
INSERT INTO Choice VALUES (103, 'Sajal', 01987654323, 'Dhaka',5);
INSERT INTO Choice VALUES (104, 'Towhid', 01987654324, 'Dhaka',4);
INSERT INTO Choice VALUES (105, 'Anchol', 01987654325, 'Dhaka',1);

SELECT * FROM Choice;
```

Results Explain Describe Saved SQL History

CUSTOMERID	CUSTOMERNAME	PHONE	CITY	FOODID
101	Fahim	1987654321	Dhaka	2
102	Anika	1987654322	Dhaka	3
103	Sajal	1987654323	Dhaka	5
104	Towhid	1987654324	Dhaka	4
105	Anchol	1987654325	Dhaka	1

Fig: Data Insertion In Choice Table

5. Address

Autocommit Display 10

```
INSERT INTO Address VALUES ('Dhaka', 6, 501);
INSERT INTO Address VALUES ('Dhaka', 9, 327);
INSERT INTO Address VALUES ('Dhaka', 13, 203);
INSERT INTO Address VALUES ('Dhaka', 3, 469);
INSERT INTO Address VALUES ('Dhaka', 17, 120);

SELECT * FROM Address;
```

Results Explain Describe Saved SQL History

CITY	ROAD	HOUSENO
Dhaka	6	501
Dhaka	9	327
Dhaka	13	203
Dhaka	3	469
Dhaka	17	120

Fig: Data Insertion In Address Table

6. DeliveryPartner

Autocommit Display 10

```
INSERT INTO DeliveryPartner VALUES(201,  'FoodPanda', 'Rahim', 'Cycle', 159951);
INSERT INTO DeliveryPartner VALUES(202,  'Pathao', 'Karim', 'Bike', 753357);
INSERT INTO DeliveryPartner VALUES(203,  'Uber', 'Babul', 'Cycle', 357753);
INSERT INTO DeliveryPartner VALUES(204,  'Foodie', 'Abul', 'Car', 951159);
INSERT INTO DeliveryPartner VALUES(205,  'SteadFast', 'Modhu', 'Van', 1793179);

SELECT * FROM DeliveryPartner;
```

Results Explain Describe Saved SQL History

DELIVERYID	COMPANYNAME	RIDERNAME	RIDERVEHICLE	PHONE
201	FoodPanda	Rahim	Cycle	159951
202	Pathao	Karim	Bike	753357
203	Uber	Babul	Cycle	357753
204	Foodie	Abul	Car	951159
205	SteadFast	Modhu	Van	1793179

Fig: Data Insertion In DeliveryPartner Table

7. Location

```
INSERT INTO Location VALUES ('Dhaka', 1215, 11);
INSERT INTO Location VALUES ('Dhaka', 1216, 7);
INSERT INTO Location VALUES ('Dhaka', 1208, 3);
INSERT INTO Location VALUES ('Dhaka', 1207, 9);
INSERT INTO Location VALUES ('Dhaka', 1209, 2);

SELECT * FROM Location;
```

Results | Explain | Describe | Saved SQL | History

CITY	ZIP	ROAD
Dhaka	1215	11
Dhaka	1216	7
Dhaka	1208	3
Dhaka	1207	9
Dhaka	1209	2

Fig: Data Insertion In Location Table

8. OrdersPending

```
INSERT INTO OrdersPending VALUES (10001,101,'9 P.M.','01-JAN-2025',250);
INSERT INTO OrdersPending VALUES (10002,102,'10 P.M.','23-JAN-2025',2000);
INSERT INTO OrdersPending VALUES (10003,103,'8.30 P.M.','19-JAN-2025',199);
INSERT INTO OrdersPending VALUES (10004,104,'7 P.M.','01-FEB-2025',600);
INSERT INTO OrdersPending VALUES (10005,105,'2 P.M.','31-JAN-2025',300);

SELECT * FROM OrdersPending;
```

Results | Explain | Describe | Saved SQL | History

ORDERID	CUSTOMERID	ORDERTIME	ORDERDATE	BILL
10001	101	9 P.M.	01-JAN-2025	250
10002	102	10 P.M.	23-JAN-2025	2000
10003	103	8.30 P.M.	19-JAN-2025	199
10004	104	7 P.M.	01-FEB-2025	600
10005	105	2 P.M.	31-JAN-2025	300

Fig: Data Insertion In OrdersPending Table

9. OrderProcess

Autocommit Display 10

```
INSERT INTO OrderProcess VALUES (10001,201,'9 P.M.','01-JAN-2025',250);
INSERT INTO OrderProcess VALUES (10002,202,'10 P.M.','23-JAN-2025',2000);
INSERT INTO OrderProcess VALUES (10003,203,'8.30 P.M.','19-JAN-2025',199);
INSERT INTO OrderProcess VALUES (10004,204,'7 P.M.','01-FEB-2025',600);
INSERT INTO OrderProcess VALUES (10005,205,'2 P.M.','31-JAN-2025',300);

SELECT * FROM OrderProcess;
```

Results Explain Describe Saved SQL History

ORDERID	DELIVERYID	ORDERTIME	ORDERDATE	BILL
10001	201	9 P.M.	01-JAN-2025	250
10002	202	10 P.M.	23-JAN-2025	2000
10003	203	8.30 P.M.	19-JAN-2025	199
10004	204	7 P.M.	01-FEB-2025	600
10005	205	2 P.M.	31-JAN-2025	300

Fig: Data Insertion In OrderProcess Table

10. Restaurant

Autocommit Display 10

```
INSERT INTO Restaurant VALUES (50001,'Khanas',963369,'Dhaka',10005);
INSERT INTO Restaurant VALUES (50002,'Kudos',147741,'Dhaka',10001);
INSERT INTO Restaurant VALUES (50003,'Steak Lounge',258852,'Dhaka',10002);
INSERT INTO Restaurant VALUES (50004,'Yam Cha',123321,'Dhaka',10004);
INSERT INTO Restaurant VALUES (50005,'Dhaba',456654,'Dhaka',10003);

SELECT * FROM Restaurant;
```

Results Explain Describe Saved SQL History

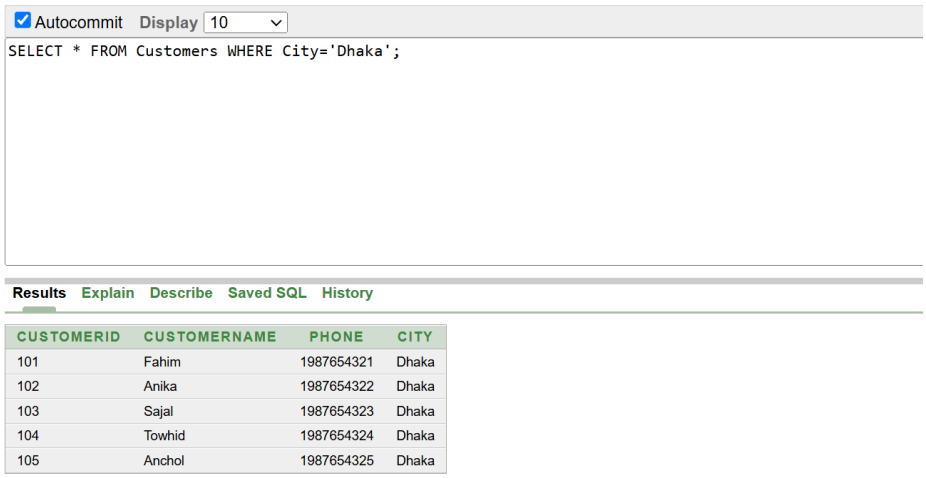
RESTAURANTID	RESTAURANTNAME	HELPLINE	CITY	ORDERID
50001	Khanas	963369	Dhaka	10005
50002	Kudos	147741	Dhaka	10001
50003	Steak Lounge	258852	Dhaka	10002
50004	Yam Cha	123321	Dhaka	10004
50005	Dhaba	456654	Dhaka	10003

Fig: Data Insertion In Restaurant Table

Query Test in DB

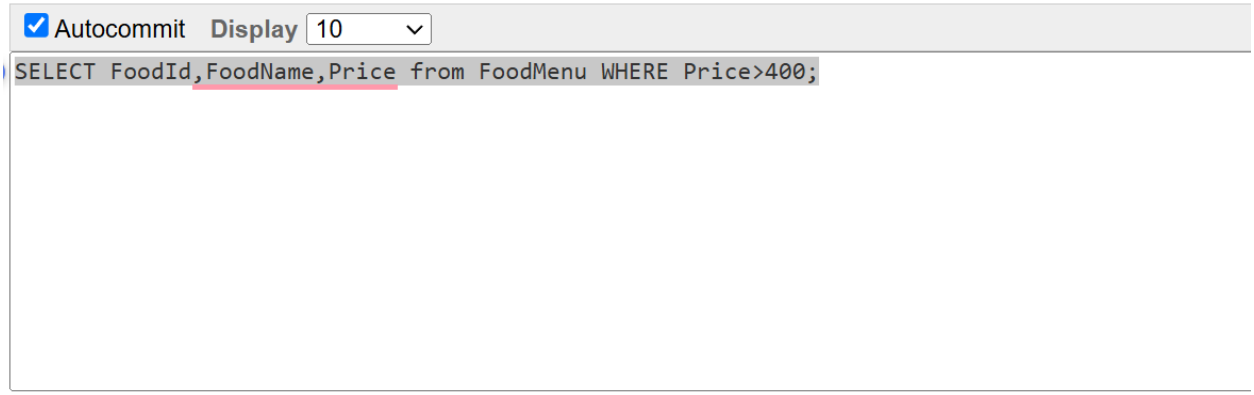
a) Basic Queries

1.



CUSTOMERID	CUSTOMERNAME	PHONE	CITY
101	Fahim	1987654321	Dhaka
102	Anika	1987654322	Dhaka
103	Sajal	1987654323	Dhaka
104	Towhid	1987654324	Dhaka
105	Anchol	1987654325	Dhaka

2.



FOODID	FOODNAME	PRICE
3	Steak	2000
4	Sushi	600

3.

Autocommit Display 10

```
SELECT Restaurantname, HelpLine FROM Restaurant;
```

Results Explain Describe Saved SQL History

RESTAURANTNAME	HELPLINE
Khanas	963369
Kudos	147741
Steak Lounge	258852
Yam Cha	123321
Dhaba	456654

4.

Autocommit Display 10

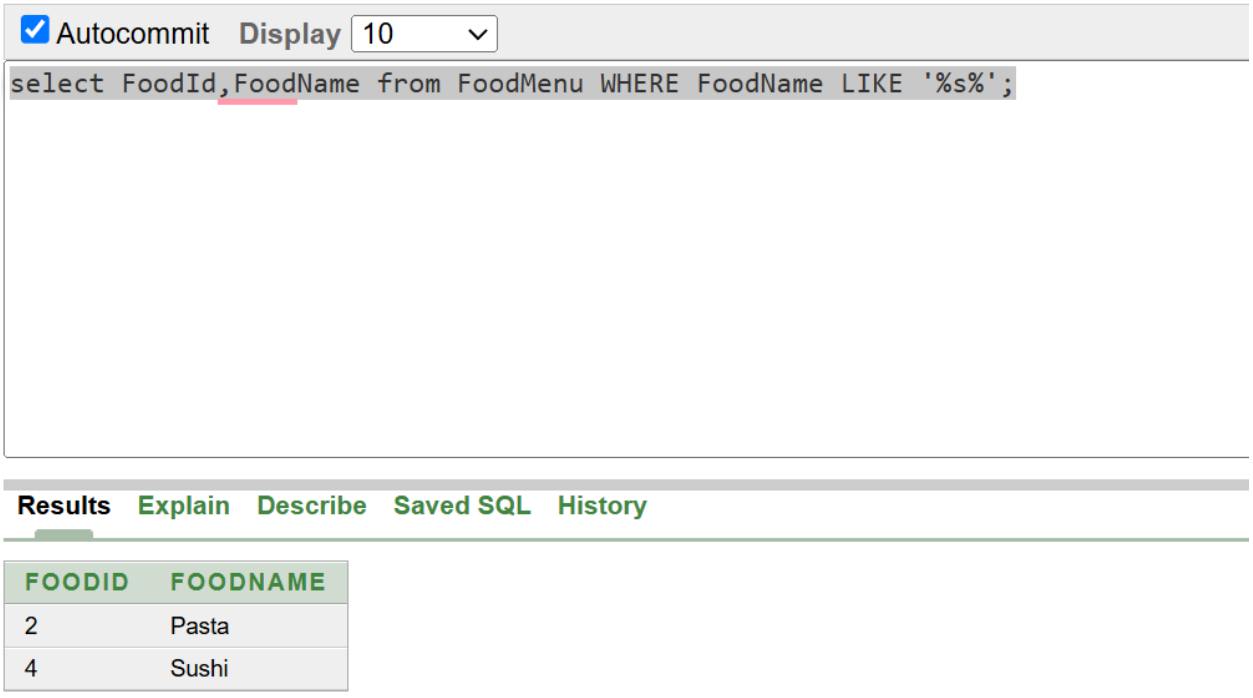
```
SELECT RiderName, RiderVehicle FROM DeliveryPartner where RiderVehicle= 'Cycle';
```

Results Explain Describe Saved SQL History

RIDERNAME	RIDERVEHICLE
Rahim	Cycle
Babul	Cycle

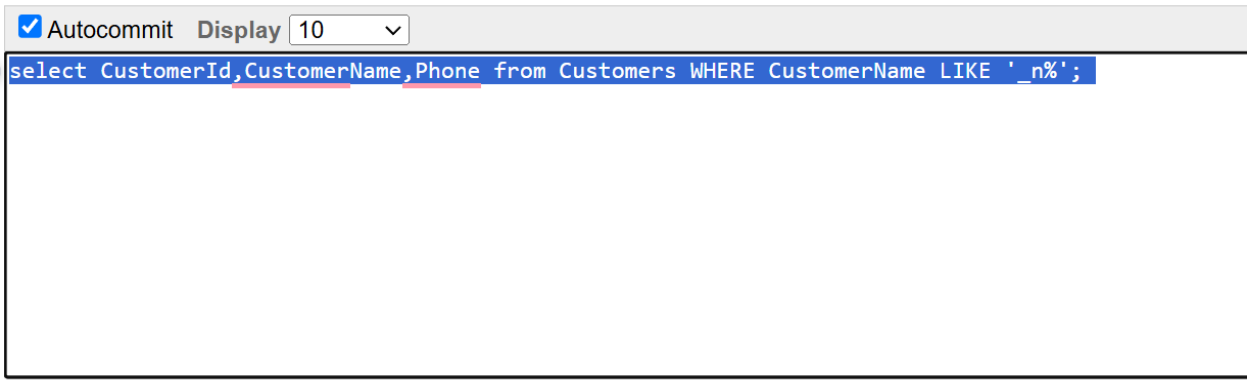
b) Two queries using the Like operator

1.



FOODID	FOODNAME
2	Pasta
4	Sushi

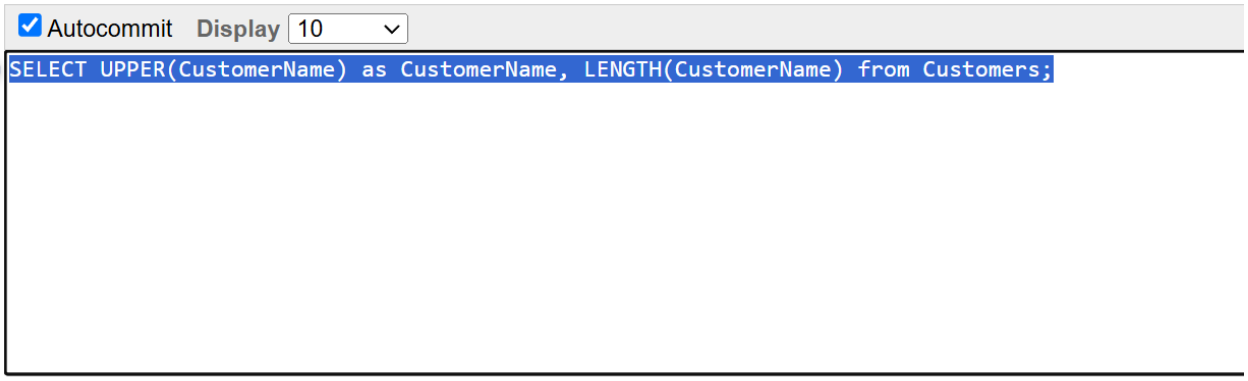
2.



CUSTOMERID	CUSTOMERNAME	PHONE
102	Anika	1987654322
105	Anchol	1987654325

c) Two queries using the Character Manipulation Functions

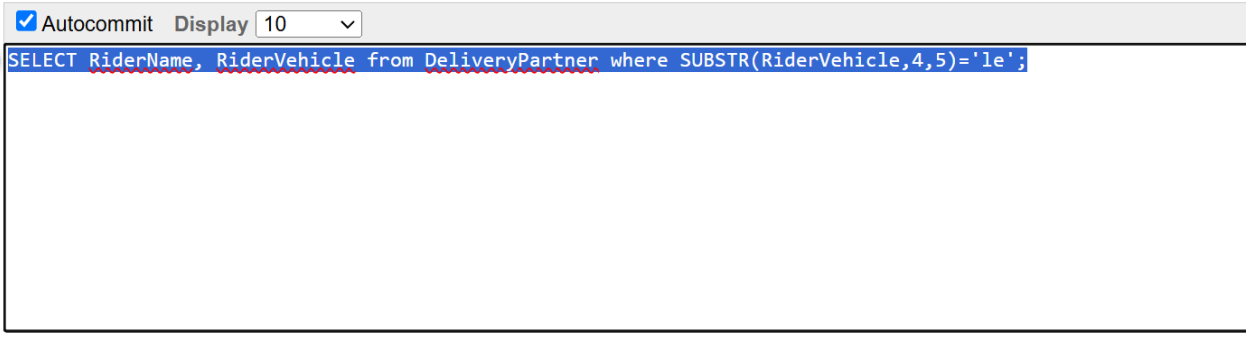
1.



Results Explain Describe Saved SQL History

CUSTOMERNAME	LENGTH(CUSTOMERNAME)
FAHIM	5
ANIKA	5
SAJAL	5
TOWHID	6
ANCHOL	6

2.

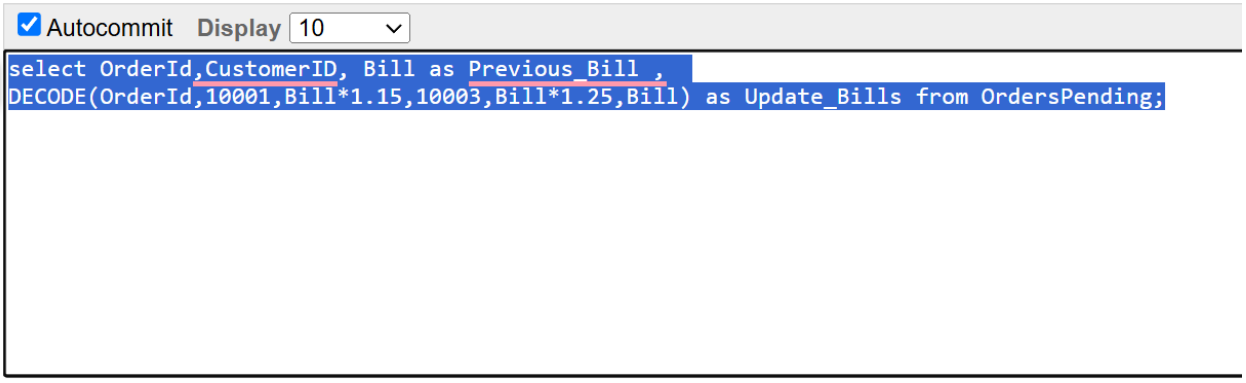


Results Explain Describe Saved SQL History

RIDERNAME	RIDERVEHICLE
Rahim	Cycle
Babul	Cycle

d) Two queries using the Decode Function

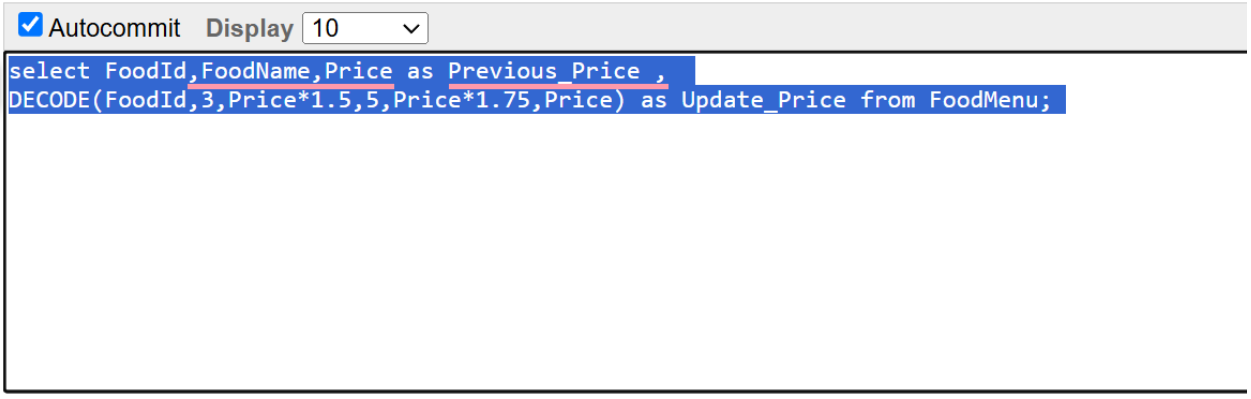
1.



Results | Explain | Describe | Saved SQL | History

ORDERID	CUSTOMERID	PREVIOUS_BILL	UPDATE_BILLS
10001	101	250	287.5
10002	102	2000	2000
10003	103	199	248.75
10004	104	600	600
10005	105	300	300

2.

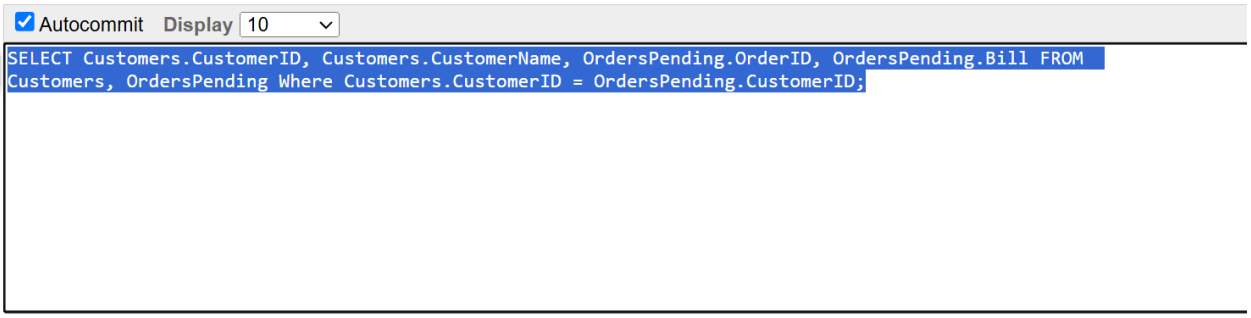


Results | Explain | Describe | Saved SQL | History

FOODID	FOODNAME	PREVIOUS_PRICE	UPDATE_PRICE
1	Burger	300	300
2	Pasta	250	250
3	Steak	2000	3000
4	Sushi	600	600
5	Vegan Bowl	199	348.25

e) 3 kinds of joining (1 query for equijoin ; 1 non-equijoin; 1 outer join)

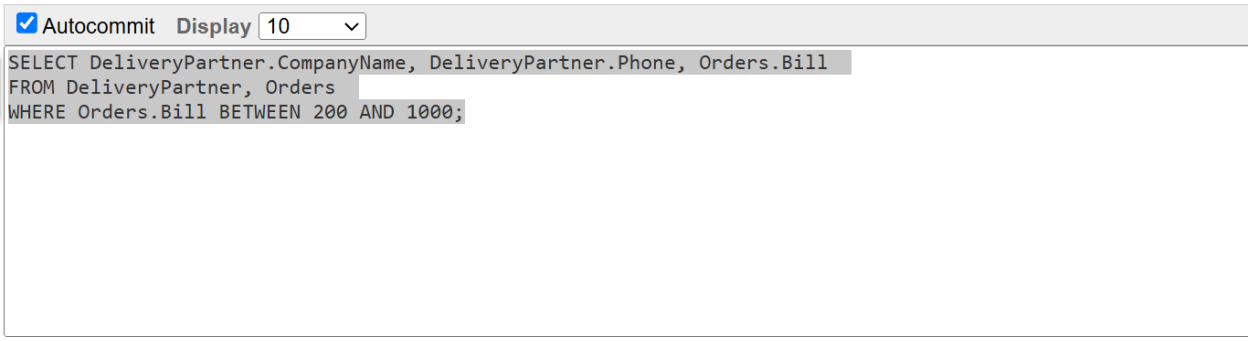
1.



Results Explain Describe Saved SQL History

CUSTOMERID	CUSTOMERNAME	ORDERID	BILL
101	Fahim	10001	250
102	Anika	10002	2000
103	Sajal	10003	199
104	Towhid	10004	600
105	Anchol	10005	300

2.



Results Explain Describe Saved SQL History

COMPANYNAME	PHONE	BILL
FoodPanda	159951	250
Pathao	753357	250
Uber	357753	250
Foodie	951159	250
SteadFast	1793179	250
FoodPanda	159951	600
Pathao	753357	600
Uber	357753	600
Foodie	951159	600
SteadFast	1793179	600

More than 10 rows available. Increase rows selector to view more rows.

3.

```
SELECT Customers.CustomerID, Customers.CustomerName, OrdersPending.OrderID, OrdersPending.Bill FROM   Customers, OrdersPending WHERE
Customers.CustomerID(+) = OrdersPending.CustomerID;
```

Results Explain Describe Saved SQL History

CUSTOMERID	CUSTOMERNAME	ORDERID	BILL
101	Fahim	10001	250
102	Anika	10002	2000
103	Sajal	10003	199
104	Towhid	10004	600
105	Anchol	10005	300

F) View

1. Create a simple view for Customers.

```
CREATE VIEW CustomersDetails AS SELECT CustomerID, CustomerName, City,Phone FROM
Customers;

desc CustomersDetails;
```

Results Explain Describe Saved SQL History

Object Type **VIEW** Object **CUSTOMERSDETAILS**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
CUSTOMERSDETAILS	CUSTOMERID	Number	-	-	-	-	-	-	-
	CUSTOMERNAME	Varchar2	20	-	-	-	-	-	-
	CITY	Varchar2	20	-	-	-	✓	-	-
	PHONE	Number	-	15	0	-	-	-	-

1 - 4

2. Create a view to show the Customer Name and the Food Name for all the food choices made by customers.

```
CREATE VIEW CustomersOrders AS SELECT c.CustomerName, f.FoodName FROM
Customers c,Choice ch,FoodMenu f where c.CustomerID = ch.CustomerID and ch.FoodID =
f.FoodID;

Desc CustomersOrders;
```

Results Explain Describe Saved SQL History

Object Type **VIEW** Object **CUSTOMERSORDERS**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
CUSTOMERSORDERS	CUSTOMERNAME	Varchar2	20	-	-	-	-	-	-
	FOODNAME	Varchar2	20	-	-	-	-	-	-

1 - 2

Description of a Successful DB connection

StudentID1: 23-55423-3

Name: Fahim Ahmed

Software Installation and JAR File Download:

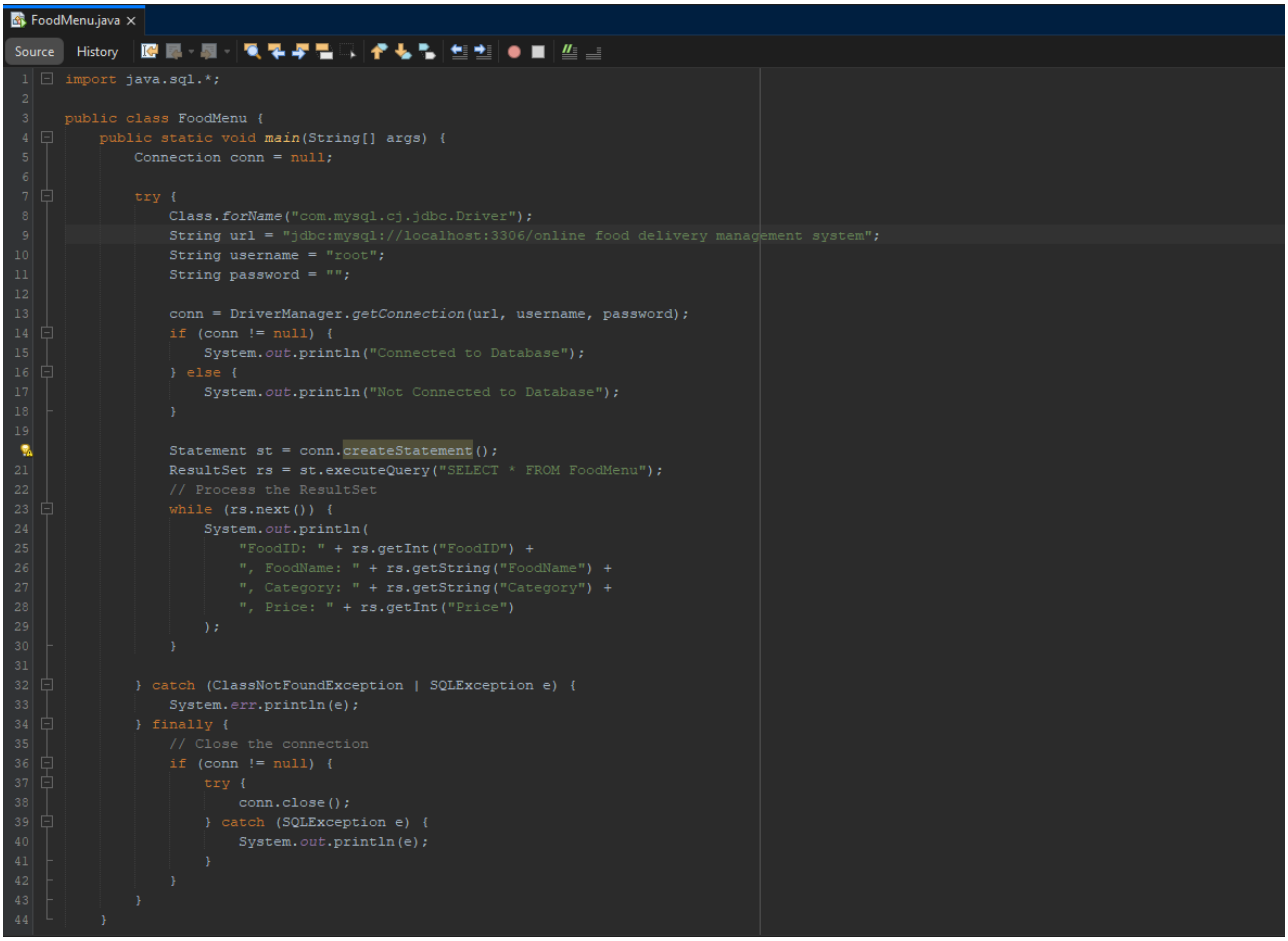
- Set up Apache NetBeans IDE 24 for editing code.
- Installed Xampp Server to create a local development setup.
- Downloaded the mysql-connector-java-8.0.30.jar file to enable database connectivity.

I started the Apache Web Server and MySQL Database using the XAMPP control panel. Next, I went to “http://localhost/phpmyadmin” to establish a new database called " Online Food Delivery Management System". Within this database, I created a table named "FoodMenu" and added columns relevant to our project. Afterwards, I inserted some data into the table.

FoodID	FoodName	Category	Price
1	Burger	Fast Food	300
2	Pasta	Casual Dining	250
3	Steak	Fine Dining	2000
4	Sushi	Ethnic Cuisine	600
5	Vegan Bowl	Specialty Cuisine	199

Fig: Data Insertion In FoodMenu Table

After setting up my database and table, I decided to utilize the NETBEANS editor for my project. I incorporated the mysql-connector-java-8.0.30.jar library into my project. Next, I wrote the code in the main file to establish a connection to the database I created with Xampp. Ultimately, I retrieved and displayed the columns from the FoodMenu table.



```java
import java.sql.*;

public class FoodMenu {
    public static void main(String[] args) {
        Connection conn = null;

        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            String url = "jdbc:mysql://localhost:3306/online food delivery management system";
            String username = "root";
            String password = "";

            conn = DriverManager.getConnection(url, username, password);
            if (conn != null) {
                System.out.println("Connected to Database");
            } else {
                System.out.println("Not Connected to Database");
            }

            Statement st = conn.createStatement();
            ResultSet rs = st.executeQuery("SELECT * FROM FoodMenu");
            // Process the ResultSet
            while (rs.next()) {
                System.out.println(
                    "FoodID: " + rs.getInt("FoodID") +
                    ", FoodName: " + rs.getString("FoodName") +
                    ", Category: " + rs.getString("Category") +
                    ", Price: " + rs.getInt("Price")
                );
            }

        } catch (ClassNotFoundException | SQLException e) {
            System.err.println(e);
        } finally {
            // Close the connection
            if (conn != null) {
                try {
                    conn.close();
                } catch (SQLException e) {
                    System.out.println(e);
                }
            }
        }
    }
```

Fig: DB Connection Code

```
run:
Connected to Database
FoodID: 1, FoodName: Burger, Category: Fast Food, Price: 300
FoodID: 2, FoodName: Pasta, Category: Casual Dining, Price: 250
FoodID: 3, FoodName: Steak, Category: Fine Dining, Price: 2000
FoodID: 4, FoodName: Sushi, Category: Ethnic Cuisine, Price: 600
FoodID: 5, FoodName: Vegan Bowl, Category: Specialty Cuisine, Price: 199
BUILD SUCCESSFUL (total time: 0 seconds)
```

Fig: Output

StudentID1: 23-55419-3

Name: Sajal Kumar Ghosh

Software Installation and JAR File Download:

- I set up Apache NetBeans IDE 24 for editing code.
- I installed XAMPP Server to establish a local development environment.
- I downloaded the mysql-connector-java-8.0.30.jar file to enable database connectivity.

I began by launching the Apache Web Server and MySQL Database using the XAMPP control panel. Then, I went to “http://localhost/phpmyadmin” to create a new database called " Online Food Delivery Management System ". Within this database, I created a table named "Customers" and added columns relevant to our project. After that, I inserted some data into the table.

CustomerID	CustomerName	Phone	City
101	Fahim	1987654321	Dhaka
102	Anika	1987654322	Dhaka
103	Sajal	1987654323	Dhaka
104	Towhid	1987654324	Dhaka
105	Anchol	1987654325	Dhaka

Fig: Data Insertion In Customers Table

After setting up my database and table, I decided to use the NETBEANS editor for my project. I included the mysql-connector-java-8.0.30.jar library in my project. Next, in the main file, I wrote the code to establish a connection to my database created with Xampp. Finally, I retrieved and displayed the columns from the Customers table.

```java
import java.sql.*;

public class Customers {
    public static void main(String[] args) {
        Connection conn = null;

        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            String url = "jdbc:mysql://localhost:3306/online food delivery management system";
            String username = "root";
            String password = "";

            conn = DriverManager.getConnection(url, username, password);
            if (conn != null) {
                System.out.println("Connected to Database");
            } else {
                System.out.println("Not Connected to Database");
            }

            Statement st = conn.createStatement();
            ResultSet rs = st.executeQuery("SELECT * FROM Customers");
            // Process the ResultSet
            while (rs.next()) {
                System.out.println(
                    "CustomerID: " + rs.getInt("CustomerId") +
                    ", CustomerName: " + rs.getString("CustomerName") +
                    ", Phone: " + rs.getInt("Phone") +
                    ", City: " + rs.getString("City")
                );
            }

        } catch (ClassNotFoundException | SQLException e) {
            System.err.println(e);
        } finally {
            // Close the connection
            if (conn != null) {
                try {
                    conn.close();
                } catch (SQLException e) {
                    System.out.println(e);
                }
            }
        }
    }
```

Fig: DB Connection Code

```
run:
Connected to Database
CustomerID: 101, CustomerName: Fahim, Phone: 1987654321, City: Dhaka
CustomerID: 102, CustomerName: Anika, Phone: 1987654322, City: Dhaka
CustomerID: 103, CustomerName: Sajal, Phone: 1987654323, City: Dhaka
CustomerID: 104, CustomerName: Towhid, Phone: 1987654324, City: Dhaka
CustomerID: 105, CustomerName: Anchol, Phone: 1987654325, City: Dhaka
BUILD SUCCESSFUL (total time: 0 seconds)
```

Fig: Output

StudentID1: 23-55070-3

Name: Anika Tabassum

Installing software and downloading JAR files:

- Set up a local development environment by installing Xampp Server.
- Installed Apache NetBeans IDE 24 for code editing.
- To enable database connectivity, the mysql-connector-java-8.0.30.jar file was downloaded.

Using the XAMPP control panel, I started the Apache Web Server and MySQL database. I then created a new database called "Online Food Delivery Management System" by navigating to "http://localhost/phpmyadmin." I make a table called "DeliveryPartner" in this freshly constructed database, and then I add columns based on our project. I then entered some data into the table.

DeliveryID	CompanyName	RiderName	RiderVehicle	Phone
201	FoodPanda	Rahim	Cycle	159951
202	Pathao	Karim	Bike	753357
203	Uber	Babul	Cycle	357753
204	Foodie	Abul	Car	951159
205	SteadFast	Modhu	Van	1793179

Fig: Data Insertion In DeliveryPartner Table

I decided to utilize the NETBEANS editor for my project after creating my database and table. In my project, I included the mysql-connector-java-8.0.30.jar library. I then added the code to connect to my database created using Xampp in the main file. At last, I obtained and printed the DeliveryPartner table's columns.

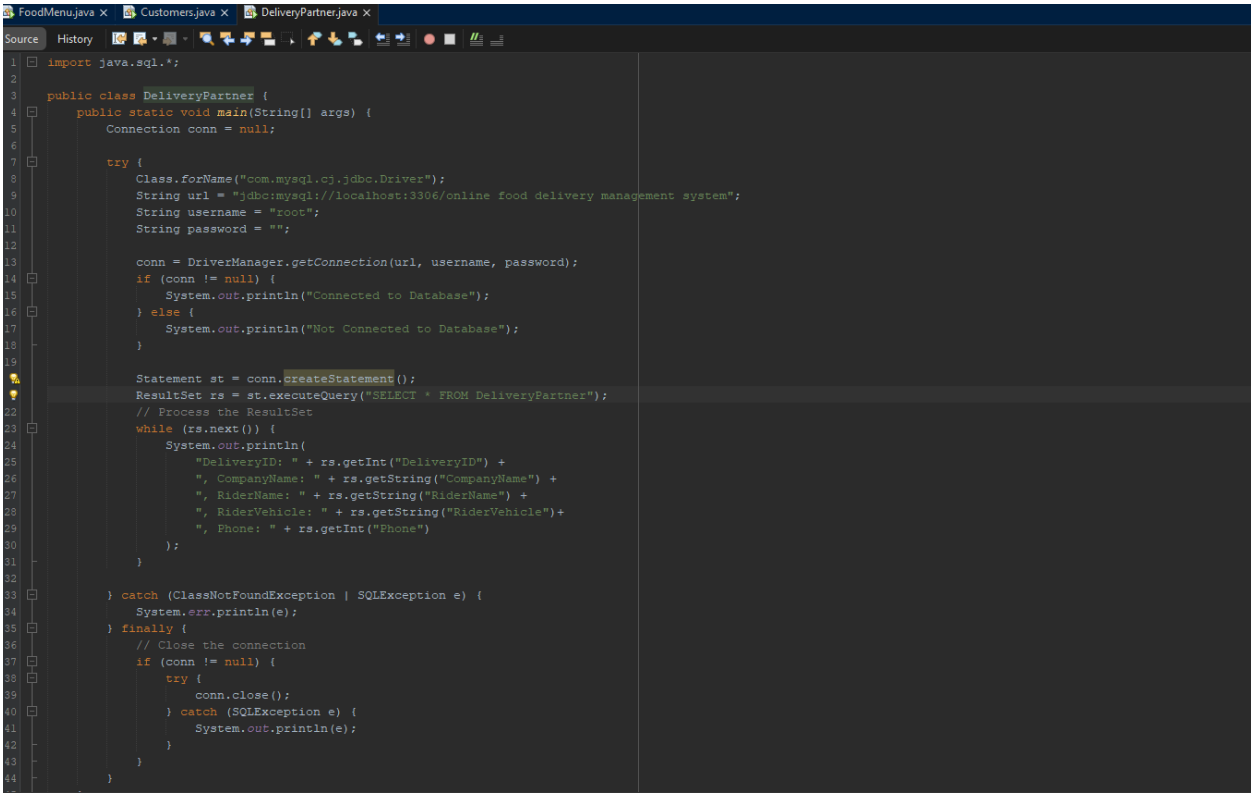


Fig: DB Connection Code

```
run:
Connected to Database
DeliveryID: 201, CompanyName: FoodPanda, RiderName: Rahim, RiderVehicle: Cycle, Phone: 159951
DeliveryID: 202, CompanyName: Pathao, RiderName: Karim, RiderVehicle: Bike, Phone: 753357
DeliveryID: 203, CompanyName: Uber, RiderName: Babul, RiderVehicle: Cycle, Phone: 357753
DeliveryID: 204, CompanyName: Foodie, RiderName: Abul, RiderVehicle: Car, Phone: 951159
DeliveryID: 205, CompanyName: SteadFast, RiderName: Modhu, RiderVehicle: Van, Phone: 1793179
BUILD SUCCESSFUL (total time: 0 seconds)
```

Fig: Output

StudentID1: 23-55036-3

Name: Md. Towhidul Islam

Software Installation and JAR File Download:

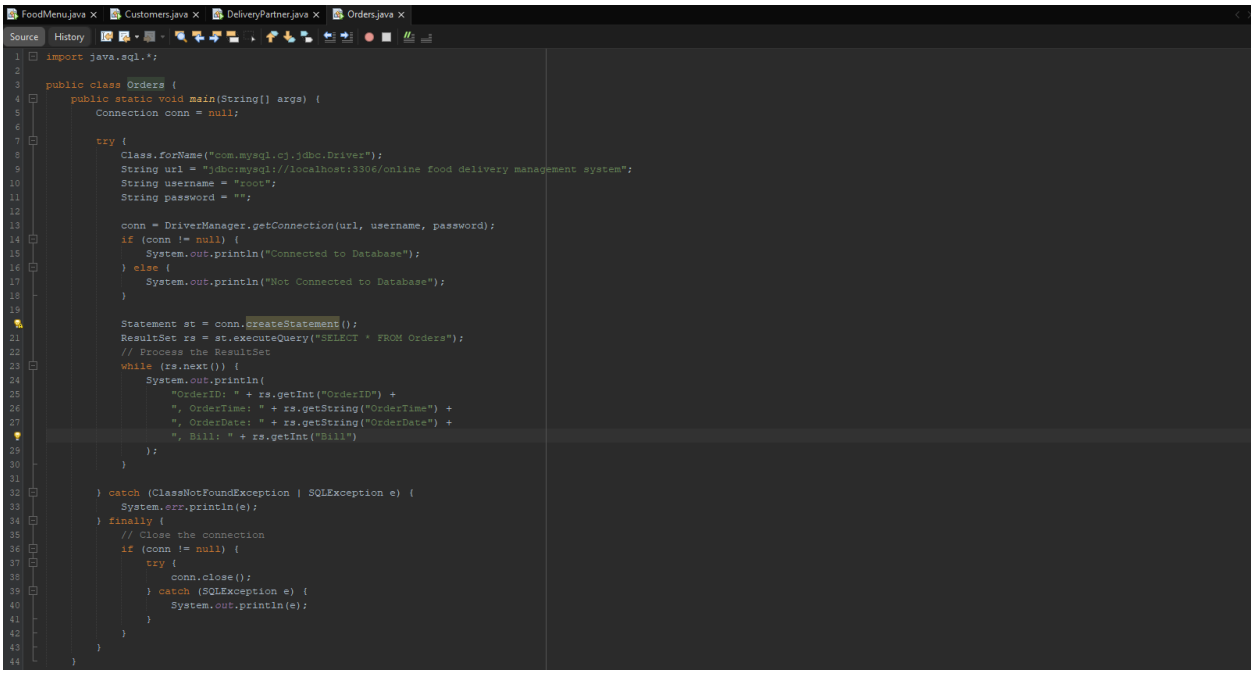
- Xampp Server was installed to create a local development environment;
- Apache NetBeans IDE 24 was installed for code editing.
- For the purpose of facilitating database connectivity, the mysql-connector-java-8.0.30.jar file was downloaded.

I started by using the XAMPP management interface to launch the Apache Web Server and MySQL Database. Next, I created a new database called "Online Food Delivery Management System" by navigating to "http://localhost/phpmyadmin." I make a "Orders" table in this newly constructed database and add columns based on our project. I then entered a few data points into the table.

OrderID	OrderTime	OrderDate	Bill
10001	9 P.M.	01-JAN-2025	250
10002	10 P.M.	23-JAN-2025	2000
10003	8.30 P.M.	19-JAN-2025	199
10004	7 P.M.	01-FEB-2025	600
10005	2 P.M.	31-JAN-2025	300

Fig: Data Insertion in Orders Table

I decided to utilize the NETBEANS editor for my project after creating my database and table. In my project, I included the mysql-connector-java-8.0.30.jar library. I then added the code to connect to my database created using Xampp in the main file. At last, I obtained and printed the Orders table's columns.



```java
import java.sql.*;

public class Orders {
    public static void main(String[] args) {
        Connection conn = null;

        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            String url = "jdbc:mysql://localhost:3306/online food delivery management system";
            String username = "root";
            String password = "";

            conn = DriverManager.getConnection(url, username, password);
            if (conn != null) {
                System.out.println("Connected to Database");
            } else {
                System.out.println("Not Connected to Database");
            }

            Statement st = conn.createStatement();
            ResultSet rs = st.executeQuery("SELECT * FROM Orders");
            // Process the ResultSet
            while (rs.next()) {
                System.out.println(
                    "OrderID: " + rs.getInt("OrderID") +
                    ", OrderTime: " + rs.getString("OrderTime") +
                    ", OrderDate: " + rs.getString("OrderDate") +
                    ", Bill: " + rs.getInt("Bill")
                );
            }

        } catch (ClassNotFoundException | SQLException e) {
            System.err.println(e);
        } finally {
            // Close the connection
            if (conn != null) {
                try {
                    conn.close();
                } catch (SQLException e) {
                    System.out.println(e);
                }
            }
        }
    }
```

Fig: DB Connection Code

```
run:
Connected to Database
OrderID: 10001, OrderTime: 9 P.M., OrderDate: 01-JAN-2025, Bill: 250
OrderID: 10002, OrderTime: 10 P.M., OrderDate: 23-JAN-2025, Bill: 2000
OrderID: 10003, OrderTime: 8.30 P.M., OrderDate: 19-JAN-2025, Bill: 199
OrderID: 10004, OrderTime: 7 P.M., OrderDate: 01-FEB-2025, Bill: 600
OrderID: 10005, OrderTime: 2 P.M., OrderDate: 31-JAN-2025, Bill: 300
BUILD SUCCESSFUL (total time: 0 seconds)
```

Fig: Output

Conclusion

The "Online Food Delivery Management System" project represents a practical and innovative solution for modern food delivery services. This system has been meticulously designed to streamline the interaction between customers, restaurants, and delivery personnel, ensuring efficiency and user satisfaction. By implementing a structured database and utilizing tools like Oracle and MySQL, this project demonstrates the application of database normalization, SQL queries, and Java connectivity in creating a reliable and functional system.

In the future, this project can be expanded to include advanced features like AI-powered recommendations for customers based on their order history, real-time GPS tracking for delivery personnel, and dynamic dashboards for restaurants to analyze their sales data. Integration with third-party payment gateways and the addition of feedback systems can further enhance user experience and trust.

The system has immense potential to benefit people by saving time, offering convenience, and promoting local businesses by providing them with a digital platform. It can also empower delivery personnel with better job opportunities and streamlined operations. Overall, this project not only shows technical skills but also addresses real-world challenges, paving the way for continuous improvements and scalability in the rapidly growing food delivery industry.